



WAYPOINT CONNECT

DATABASE / DATA MODEL DOCUMENTATION

Database / Data Model

Models, relationships, constraints and ER diagram from
prisma/schema.prisma

Project WaypointConnect — delivery planning and tracking for Waypoint Group's Fresh, Style and Tech brands

Event Tech-Triathlon 2026 · Team Dark Horse

Source Generated 2026-10-04 · repository commit c1e3eb6 + working tree

Accuracy rule: every statement is taken from the project files. Anything that could not be verified is marked "Not verified from the provided project files". No passwords or secrets are included.

Contents

1. Database overview	2
1.1 Database overview	2
2. Enum values	3
2.1 Enum values	3
3. Models	4
3.1 All models (tables)	4
4. Relationships	9
4.1 Relationships	9
5. ER diagram	10
6. Constraints, indexes and rules	12
6.1 Important constraints	12
6.2 Important indexes	12
6.3 Database business rules	12

1. Database overview

1.1 Database overview

The system stores all state in one PostgreSQL 16 database named `waypoint`, defined by `prisma/schema.prisma` (Prisma 6, `provider = "postgresql"`, connection from `DATABASE_URL`). The schema contains **19 models** and **14 enums**. Reference data (outlets, vehicles) is loaded from the competition CSVs by `prisma/seed.ts`; orders, plans and everything operational are created by the application. Dates that are calendar days use `@db.Date`; timestamps are stored in UTC.

- **How the schema reaches the database:** the API container runs `prisma db push` at every start (not migrations). An untracked `prisma/migrations/20261004071614_init` folder exists, but it creates only 17 tables (no `Driver` or `Setting`) and its `Role` enum lacks `admin`, so it does not match the current schema.
- **Primary keys:** string ids (`OUT001`, `VEH001`, `DRV001`) for reference data and cuid for generated rows; `DeliveryEvent` uses the client-generated UUID; two tables use composite or natural keys (`VehicleDayStatus`, `Setting`).
- **Not in the schema:** no many-to-many relations are declared; three tables (`PlanVersion`, `PlanAck`, `LoadMove`) refer to a plan by a plain `planId` column without a relation.

2. Enum values

2.1 Enum values

Enum	Values	Used by
Role	dispatcher loader driver store admin	User.role, PlanAck.role, Notification.role
Brand	Fresh Style Tech	Outlet.brand, Trip.brand
Temp	ambient chilled	Order.temp
DockType	rear_dock street mall_bay	Outlet.dockType
Parking	normal van_only mall_dock	Outlet.parking
Depot	Peliyagoda Kandy	Outlet.depot, Vehicle.depot, Plan.depot, User.depot
VehicleType	truck van	Vehicle.type
VehicleTemp	reefer ambient	Vehicle.temp
OrderStatus	placed locked planned deferred loaded out_for_delivery delivered failed received cancelled	Order.status
PlanStatus	draft published	Plan.status
DeferralReason	no_reefer_capacity vehicle_full time_window fuel_quota no_vehicle other	Deferral.reason
DeferralType	unavoidable choice	Deferral.type
StopStatus	pending arrived delivered failed	TripStop.status
LoadStatus	to_load loading sealed	Trip.loadStatus

3. Models

3.1 All models (tables)

Optional fields are marked with `?`. "Relationships" lists foreign keys on the model and the inverse sides declared by Prisma.

Model: Outlet table "Outlet"	
Purpose	A store that orders goods (reference data from <code>data/outlets.csv</code>).
Primary key	<code>id</code> (string, e.g. OUT001)
Important fields	<code>brand</code> , <code>district</code> , <code>depot</code> , <code>dockType</code> , <code>parking</code> , <code>mallWindow?</code> , <code>windowOpen</code> / <code>windowClose</code> (HH:MM), <code>lastServedOn?</code> , <code>managerName?</code> , <code>managerPhone?</code> , <code>deliveryNote?</code>
Relationships	One-to-many: <code>orders</code> (Order), <code>users</code> (store-manager User)
Business purpose	Defines where, when and how goods can be delivered; drives van-only, depot and time-window rules and fairness (last served).

Model: Vehicle table "Vehicle"	
Purpose	A truck or van (from <code>data/vehicles.csv</code>).
Primary key	<code>id</code> (e.g. VEH001)
Important fields	<code>type</code> , <code>temp</code> (reefer/ambient), <code>weightCapKg</code> , <code>volumeCapM3</code> , <code>depot</code> , <code>kmPerL</code> , <code>weeklyFuelQuotaL</code> , <code>fuelUsedWeekL</code> , <code>driverName?</code> (copy), <code>driverId?</code> (unique)
Relationships	FK <code>driverId</code> → Driver (one-to-one, optional). One-to-many: <code>dayStatus</code> , <code>trips</code> , <code>users</code>
Business purpose	The capacity the planning engine allocates; the vehicle account is how a driver signs in.

Model: Driver table "Driver"	
Purpose	A person who drives; the admin assigns at most one driver per vehicle.
Primary key	<code>id</code> (e.g. DRV001)
Important fields	<code>name</code> , <code>phone?</code> , <code>licenseNo?</code> , <code>createdAt</code>
Relationships	Inverse side of one-to-one with Vehicle (<code>vehicle?</code>)
Business purpose	Lets the administrator manage who drives which vehicle; 60 drivers plus 6 spares are seeded.

Model: VehicleDayStatus table "VehicleDayStatus"

Purpose	Workshop days. No row means the vehicle is available.
Primary key	Composite (<code>vehicleId</code> , <code>date</code>)
Important fields	<code>status</code> (in_workshop), <code>note?</code> , <code>setById?</code>
Relationships	FK <code>vehicleId</code> → Vehicle (many-to-one)
Business purpose	Vehicles in the workshop are refused by the engine (<code>VEHICLE_UNAVAILABLE</code>).

Model: User table "User"

Purpose	A sign-in account of any role.
Primary key	<code>id</code> (cuid)
Important fields	<code>email</code> (unique), <code>name</code> , <code>passwordHash</code> , <code>role</code> , <code>depot?</code> , <code>outletId?</code> , <code>vehicleId?</code> , <code>active</code> , <code>customPassword</code> , <code>lastLoginAt?</code> , <code>createdAt</code>
Relationships	FK <code>outletId</code> → Outlet (optional), FK <code>vehicleId</code> → Vehicle (optional)
Business purpose	Authentication and scoping: store accounts carry an outlet, driver accounts a vehicle, loaders and fixed-depot dispatchers a depot; <code>active=false</code> switches an account off.

Model: Order table "Order"

Purpose	A store order for one delivery date and one temperature class.
Primary key	<code>id</code> (cuid)
Important fields	<code>outletId</code> , <code>deliveryDate</code> , <code>temp</code> , <code>units</code> , <code>weightKg</code> , <code>volumeM3</code> , <code>status</code> , <code>note?</code> , <code>placedAt</code> , <code>sourceRef?</code> (e.g. S1-003, <code>carry:</code> , <code>short:</code>)
Relationships	FK <code>outletId</code> → Outlet. One-to-one: <code>stop</code> (TripStop?), <code>receipt</code> (Receipt?). One-to-many: <code>deferrals</code> , <code>loadChecks</code> , <code>lines</code> . Index (<code>deliveryDate</code> , <code>status</code>)
Business purpose	The unit of planning: never split; placed before/after cutoff by <code>placedAt</code> ; carried-over and short orders reference their origin in <code>sourceRef</code> .

Model: OrderLine table "OrderLine"

Purpose	A goods line of an order, counted in packs.
Primary key	<code>id</code> (cuid)
Important fields	<code>orderId</code> , <code>seq</code> , <code>product</code> , <code>pack</code> , <code>packSize</code> , <code>packs</code> , <code>ordered</code> , <code>weightKg</code> , <code>loadedAt?</code> , <code>shortPacks</code> , <code>problem?</code>
Relationships	FK <code>orderId</code> → Order (many-to-one, cascade delete). Index <code>orderId</code>
Business purpose	What loaders tick, drivers deliver and stores confirm; packs on all lines add up to <code>Order.units</code> (enforced by <code>goods.ts</code> , not by the database).

Model: Plan table "Plan"

Purpose	The plan for one depot on one date.
Primary key	<code>id</code> (cuid)
Important fields	<code>date</code> , <code>depot</code> , <code>status</code> (draft/published), <code>version</code> (0 until first published), <code>publishedAt?</code> , <code>options</code> (Json)
Relationships	One-to-many: <code>trips</code> , <code>deferrals</code> . Unique (<code>date</code> , <code>depot</code>)
Business purpose	Holds the dispatcher's plan; loaders and drivers only work from a plan with <code>version > 0</code> ; a new version makes drivers refetch.

Model: Trip table "Trip"

Purpose	One trip of one vehicle (1 or 2 per day) for one brand and district.
Primary key	<code>id</code> (cuid)
Important fields	<code>planId</code> , <code>vehicleId</code> , <code>tripNo</code> , <code>brand</code> , <code>district</code> , <code>departAt</code> , <code>minutes</code> , <code>loadStatus</code> , <code>bay</code> , <code>loaderId?</code> , <code>loaderName?</code> , <code>loadingStartedAt?</code> , <code>sealedAt?</code> , <code>sealNo?</code> , <code>reeferTempC?</code> , <code>heldAt?</code> , <code>departedAt?</code>
Relationships	FK <code>planId</code> → Plan (cascade), FK <code>vehicleId</code> → Vehicle. One-to-many: <code>stops</code> , <code>loadChecks</code> . Unique (<code>planId</code> , <code>vehicleId</code> , <code>tripNo</code>)
Business purpose	Encodes max 2 trips per vehicle, one brand and one district per trip; <code>sealedAt</code> / <code>departedAt</code> lock the trip.

Model: TripStop table "TripStop"

Purpose	An order placed on a trip, in delivery sequence.
Primary key	<code>id</code> (cuid)
Important fields	<code>tripId</code> , <code>orderId</code> (unique), <code>seq</code> , <code>plannedArrival</code> , <code>status</code> , <code>arrivedAt?</code> , <code>deliveredAt?</code> , <code>deliveredUnits?</code> , <code>receiverName?</code> , <code>failedReason?</code>
Relationships	FK <code>tripId</code> → Trip (cascade), FK <code>orderId</code> → Order (one-to-one). One-to-many: <code>events</code> (DeliveryEvent)
Business purpose	Unique <code>orderId</code> means an order is on at most one stop, so orders are never split.

Model: Deferral table "Deferral"

Purpose	A record that an order was not served on its run, with the reason.
Primary key	<code>id</code> (cuid)
Important fields	<code>orderId</code> , <code>planId?</code> , <code>reason</code> , <code>type</code> , <code>note?</code> , <code>detail?</code> , <code>newDate</code> , <code>byUserId?</code> , <code>confirmedAt?</code> , <code>storeResponse?</code> (accepted cancelled), <code>respondedAt?</code> , <code>createdAt</code>
Relationships	FK <code>orderId</code> → Order, FK <code>planId</code> → Plan (optional). Index <code>orderId</code>
Business purpose	Every deferral has a reason and is <code>unavoidable</code> or <code>choice</code> ; a second deferral needs <code>confirmedAt</code> before publishing; the store answers on SM6.

Model: LoadCheck table "LoadCheck"

Purpose	A loader's problem report on a goods line (L3).
Primary key	<code>id</code> (cuid)
Important fields	<code>tripId</code> , <code>orderId</code> , <code>lineId?</code> , <code>loadedUnits</code> , <code>shortUnits</code> , <code>problem?</code> , <code>action?</code> (<code>carry_over replace hold</code>), <code>note?</code> , <code>photo?</code> , <code>byUserId?</code> , <code>byName?</code> , <code>at</code>
Relationships	FK <code>tripId</code> → Trip (cascade), FK <code>orderId</code> → Order. <code>lineId</code> is a plain column (no relation)
Business purpose	Records what went wrong at the dock; short packs are carried to the next run when the trip is sealed.

Model: DeliveryEvent table "DeliveryEvent"

Purpose	One client event from the driver app (offline-first).
Primary key	<code>eventId</code> (client UUID)
Important fields	<code>stopId</code> , <code>type</code> (<code>stop.arrived stop.delivered stop.failed</code>), <code>occurredAt</code> , <code>receivedAt</code> , <code>deviceId</code> , <code>receiverName?</code> , <code>deliveredUnits?</code> , <code>reason?</code> , <code>note?</code> , <code>photo?</code> , <code>signature?</code> , <code>outcome?</code>
Relationships	FK <code>stopId</code> → TripStop (many-to-one)
Business purpose	The primary key makes <code>POST /api/sync</code> idempotent: a repeated event id is skipped.

Model: PlanVersion table "PlanVersion"

Purpose	What each published version of a plan looked like.
Primary key	<code>id</code> (cuid)
Important fields	<code>planId</code> , <code>version</code> , <code>snapshot</code> (Json), <code>publishedAt</code> , <code>byName?</code> ; unique (<code>planId</code> , <code>version</code>)
Relationships	<code>planId</code> is a plain column: no Prisma relation to Plan
Business purpose	Lets the dock see "v1 → v2" changes.

Model: PlanAck table "PlanAck"

Purpose	Who confirmed they work from which plan version.
Primary key	<code>id</code> (cuid)
Important fields	<code>planId</code> , <code>version</code> , <code>userId</code> , <code>userName</code> , <code>role</code> , <code>at</code> ; unique (<code>planId</code> , <code>version</code> , <code>userId</code>)
Relationships	Plain columns, no relations
Business purpose	The dispatcher sees who acknowledged a plan change and when.

Model: LoadMove table "LoadMove"

Purpose	A loader moved an order's goods to its new vehicle after a plan change (L4 "Mark moved").
Primary key	<code>id</code> (cuid)
Important fields	<code>planId</code> , <code>version</code> , <code>orderId</code> , <code>byName?</code> , <code>movedAt</code> ; unique (<code>planId</code> , <code>version</code> , <code>orderId</code>)
Relationships	Plain columns, no relations
Business purpose	Tracks the physical re-loading after a republish.

Model: Receipt table "Receipt"

Purpose	The store manager's confirmation of what arrived (SM5).
Primary key	<code>id</code> (cuid)
Important fields	<code>orderId</code> (unique), <code>confirmed</code> , <code>issue?</code> , <code>issueUnits?</code> , <code>note?</code> , <code>lines</code> (Json: ok short damaged per line), <code>deliveredAt?</code> , <code>byName?</code> , <code>at</code>
Relationships	FK <code>orderId</code> → Order (one-to-one)
Business purpose	Closes the loop; a short or damaged line raises a "Store issue" alert for the dispatcher.

Model: Notification table "Notification"

Purpose	Messages to stores and alerts for the dispatcher.
Primary key	<code>id</code> (cuid)
Important fields	<code>role?</code> , <code>outletId?</code> , <code>userId?</code> , <code>kind</code> , <code>title?</code> , <code>message</code> , <code>refs</code> (Json), <code>read</code> , <code>done</code> , <code>createdAt</code> . Indexes (<code>outletId</code> , <code>read</code>), (<code>role</code> , <code>createdAt</code>)
Relationships	No relations (plain columns)
Business purpose	Store notifications (plan published, deferrals, deliveries) and dispatcher alerts (Live Tracking).

Model: Setting table "Setting"

Purpose	Small system settings set by the admin, e.g. the <code>demo</code> key.
Primary key	<code>key</code> (string)
Important fields	<code>value</code> (Json), <code>updatedAt</code>
Relationships	None
Business purpose	Persists Demo mode state (on/off, day, clocks) across API restarts.

4. Relationships

4.1 Relationships

One-to-one

Relationship	How it is enforced
Vehicle ↔ Driver (optional on both sides)	<code>Vehicle.driverId</code> is <code>@unique</code> : a driver is never on two vehicles
Order ↔ TripStop (an order has 0 or 1 stop)	<code>TripStop.orderId</code> is <code>@unique</code> : orders are never split
Order ↔ Receipt (0 or 1)	<code>Receipt.orderId</code> is <code>@unique</code>

One-to-many

"One" side	"Many" side (FK)	Notes
Outlet	<code>Order.outletId</code>	
Outlet	<code>User.outletId</code>	optional: store-manager accounts
Vehicle	<code>User.vehicleId</code>	optional: driver (vehicle) accounts
Vehicle	<code>Trip.vehicleId</code>	
Vehicle	<code>VehicleDayStatus.vehicleId</code>	workshop days
Plan	<code>Trip.planId</code>	cascade delete
Plan	<code>Deferral.planId</code>	optional
Trip	<code>TripStop.tripId</code>	cascade delete
Trip	<code>LoadCheck.tripId</code>	cascade delete
Order	<code>OrderLine.orderId</code>	cascade delete
Order	<code>Deferral.orderId</code>	
Order	<code>LoadCheck.orderId</code>	
TripStop	<code>DeliveryEvent.stopId</code>	

Many-to-many

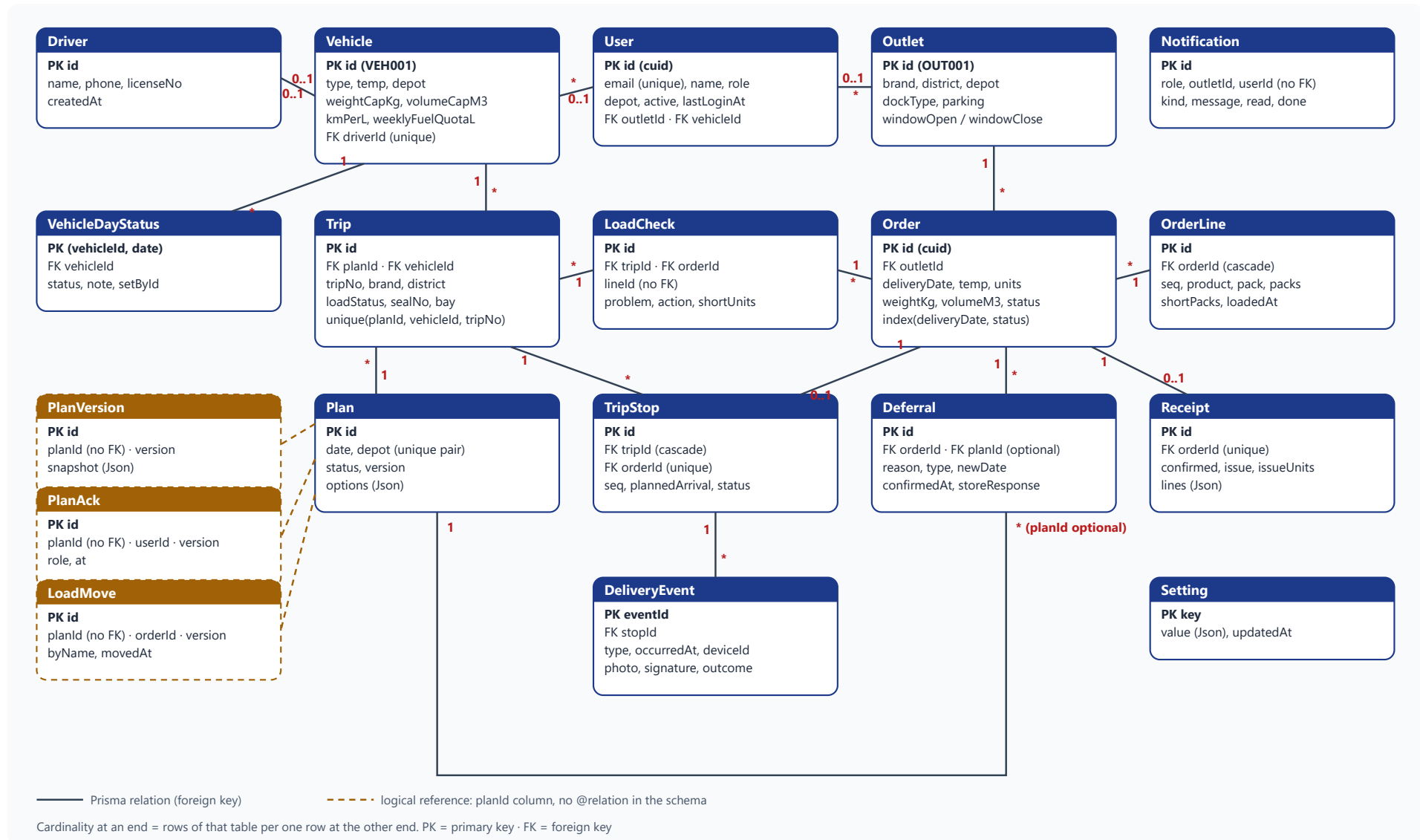
None are declared. An order reaches a trip through `TripStop` (Trip 1—* TripStop, Order 1—0..1 TripStop), which behaves as a join table that allows each order on at most one trip.

Logical references without a foreign key

These columns hold an id but the schema declares no relation, so the database does not enforce them:

`PlanVersion.planId`, `PlanAck.planId / userId`, `LoadMove.planId / orderId`, `LoadCheck.lineId / byUserId`, `Deferral.byUserId`, `Trip.loaderId`, `VehicleDayStatus.setById`, `Notification.outletId / userId`. The older [docs/data-model.md](#) draws PlanVersion, PlanAck and LoadMove as relations of Plan; in the Prisma schema they are logical references (dashed in the ER diagram).

5. ER diagram



Entity-relationship diagram drawn only from `prisma/schema.prisma`: 19 models. Solid lines are Prisma relations; dashed lines are logical planId references without a relation.

6. Constraints, indexes and rules

6.1 Important constraints

Constraint	Model	Effect
@unique email	User	One account per email address
@unique driverId	Vehicle	One driver per vehicle and never two vehicles for one driver (also guaranteed by the admin API)
@unique orderId	TripStop, Receipt	An order has at most one stop and one receipt
@@unique([date, depot])	Plan	One plan per depot per day
@@unique([planId, vehicleId, tripNo])	Trip	No duplicate trip numbers for a vehicle in a plan
@@unique([planId, version])	PlanVersion	One snapshot per published version
@@unique([planId, version, userId])	PlanAck	A user acknowledges a version once
@@unique([planId, version, orderId])	LoadMove	An order is marked moved once per version
@@id([vehicleId, date])	VehicleDayStatus	One workshop status per vehicle per day
@id eventId	DeliveryEvent	Idempotent sync of offline events
onDelete: Cascade	OrderLine→Order, Trip→Plan, TripStop→Trip, LoadCheck→Trip	Deleting a parent removes its children (used when demo data is reset)

6.2 Important indexes

Index	Model	Supports
(deliveryDate, status)	Order	Loading a delivery day's orders by status
(orderId)	OrderLine, Deferral	Fetching lines and deferrals of an order
(outletId, read)	Notification	Unread count for a store
(role, createdAt)	Notification	Dispatcher alert feed
Unique indexes above		Each @unique / @@unique creates an index

6.3 Database business rules

- Orders are never split: `TripStop.orderId` is unique.
- A plan is per depot and per date; loaders and drivers use only published plans (`version > 0`).
- Every deferral stores a reason (`DeferralReason`) and a type (`unavoidable` or `choice`).
- A sealed or departed trip is identified by `sealedAt` / `departedAt` and is locked by the API (not by a database constraint).

- Packs on an order's lines add up to `Order.units` : an application rule (`goods.ts` , tested), not a database constraint.
- Each account has its own password hash; `active=false` switches the account off; `customPassword=true` stops the seed from resetting an admin-set password.
- The database stays in UTC; dates used for planning are `@db.Date` values interpreted as Sri Lanka calendar days by the application.